



《人工智能综合设计》—信息检索和倒排索引

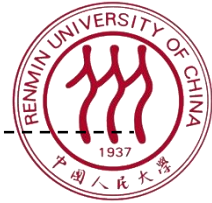


人工智能综合设计 教研组



今日目标

- 实现倒排索引
- 基于倒排索引，支持两种布尔查询：
 - 逻辑与AND
 - 逻辑或OR
- 在昨天抓取的网页上建立倒排索引



提纲



人工智能综合设计03 信息检索和倒排索引

- ☐ 信息检索简介
- ☐ 词-文档矩阵
- ☐ 倒排索引
- ☐ 查询处理



信息检索

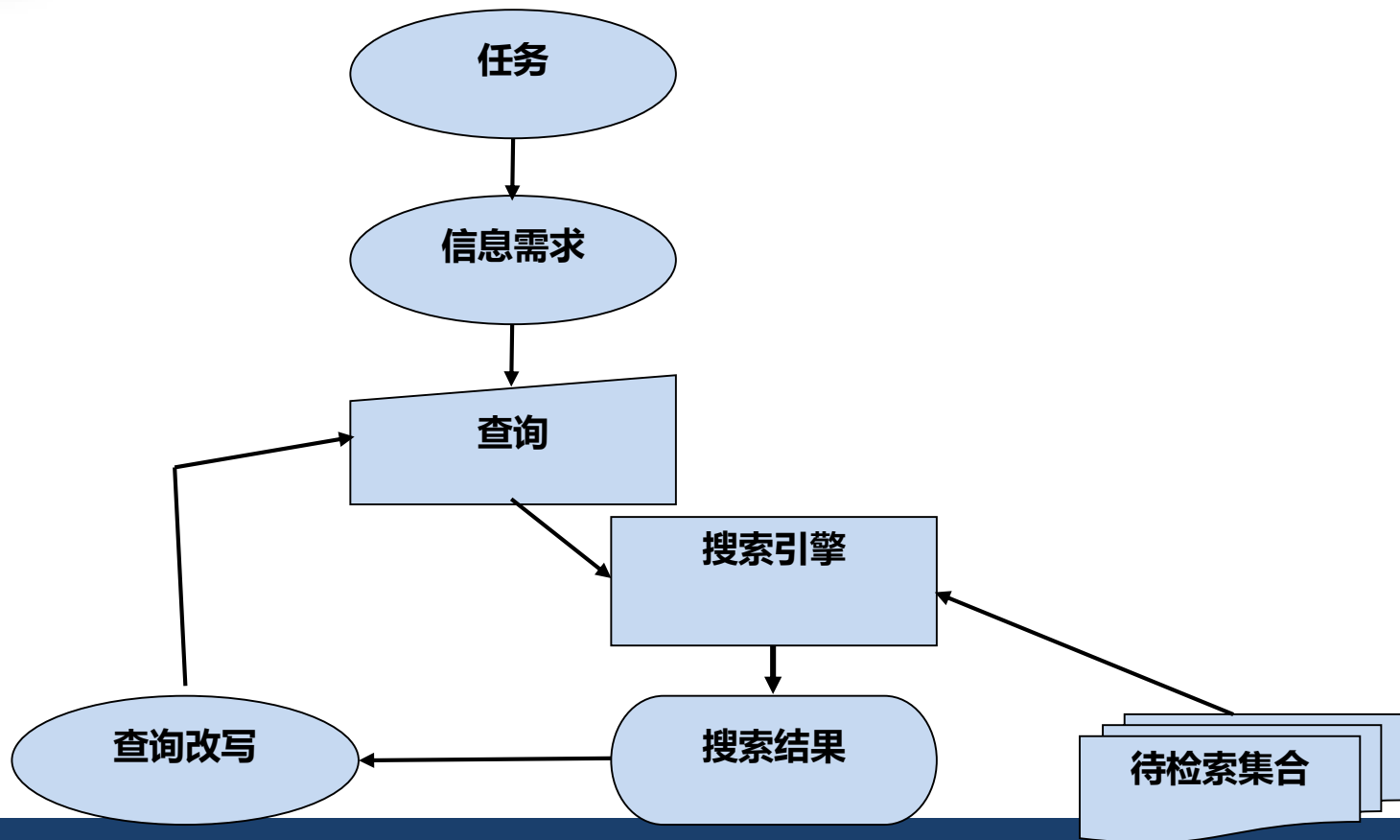
- 信息检索 (Information Retrieval) : 在**非结构化信息构成的大规模数据集**中查找能够满足用户**信息需求的内容**
 - **非结构化信息**: 通常是文本形式的 (与数据库中保存的结构化信息相对应)
 - **大规模待检索集合**: (保存在计算机中的) 网页集合/邮件集合/本地文档集合
 - **信息需求**: 通常用户会用查询来表达其信息需求
 - **内容**: 文本文档/网页
- 常见的信息检索系统:
 - 网络搜索引擎
 - 图书馆信息检索系统
 - 邮件搜索
 - 聊天记录搜索
 -

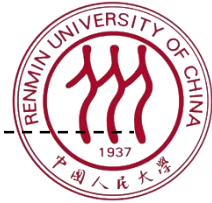


信息检索的基本假设

- 待检索集合 (Collection) :
 - 包含很多待检索的文档的集合
 - 可以暂时假设该集合是**静态的**
- 信息检索的目标:
 - 返回包含与用户**信息需求相关** (relevant) 信息的文档, 以帮助用户完成各种**任务**

一个经典的搜索过程的模型





提纲



人工智能综合设计03 信息检索和倒排索引

- □ 信息检索简介
- □ 词-文档矩阵
- □ 倒排索引
- □ 查询处理



高瓴人工智能学院老师们的研究方向

- 哪位老师研究**信息检索**
- 哪位老师研究**计算机视觉**
- 哪位老师研究**自然语言处理**和**推荐系统**
- 哪位老师研究**信息检索**和**搜索**
- 抓取老师们的个人主页信息，对其进行分析和检索！
 - <https://gsai.ruc.edu.cn/addons/teacher/index.html>



高瓴人工智能学院老师们的研究方向



您所在的位置: 首页 - 师资队伍 - 师资队伍

师资队伍



文继荣 教授 (长聘教授)

中国人民大学高瓴人工智能学院执行院长, 信息管理与分析方法研究北京市重点实验室主任。高瓴人工智能研究院首席科学家, 北京市第十三届政协党外知识分子建言献策专家组专家, 入选首批

中文

EN



卢志武 教授 (长聘副教授)

卢志武博士, 中国人民大学高瓴人工智能学院2005年7月毕业于北京大学数学科学学院信息科学学位; 2011年3月毕业于香港城市大学计算机系。主要研究方向包括机器学习、计算机视觉等...

中文

EN



文继荣

教授 (长聘教授)

中国人民大学高瓴人工智能学院执行院长, 信息学院院长, 大数据管理与分析方法研究北京市重点实验室主任。文继荣担任北京智源人工智能研究院首席科学家, 北京市第十三届政协委员, 中央统战部党外知识分子建言献策专家组专家, 入选首批“北京高校卓越青年科学家计划项目”。曾任微软亚洲研究院高级研究员和互联网搜索与挖掘组主任。文继荣长期从事大数据和人工智能领域的研究, 已在信息检索、数据挖掘、机器学习、数据库等领域国际著名学术会议和期刊上发表论文200余篇, 总计引用15000余次, H-Index为57。担任AIRS 2016大会

教育经历

- 1996 ~ 1999, 中国科学院计算技术研究所 博士研究生
- 1994 ~ 1996, 中国人民大学信息学院 硕士研究生
- 1990 ~ 1994, 中国人民大学信息学院 本科生

工作经历

- 2013 ~ 至今, 中国人民大学信息学院 教授
- 1999 ~ 2013, 微软亚洲研究院 历任副研究员、研究员、项目负责人、主任研究员、高级研究员

研究方向

互联网大数据管理、信息检索 (互联网搜索), 数据挖掘和机器学习, 尤其关注跨领域的研究和大规模数据管理系统的开发。

学生要求

对学生的培养要求



词-文档矩阵

	文继荣	窦志成	卢志武	徐君	魏哲巍	赵鑫	宋睿华	严睿	许洪腾	苏冰	王子贺	沈蔚然	胡迪	刘勇	毛佳昕	陈旭
信息检索	1	1	0	1	0	1	1	1	0	0	0	0	0	0	1	1
搜索	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	0
推荐	0	0	0	1	0	1	1	0	0	0	0	0	0	0	0	1
数据库	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
语言	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0
视觉	0	0	1	0	0	0	0	0	0	1	0	0	1	0	0	0
对话	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0
数据挖掘	1	1	0	1	0	1	0	0	0	0	0	0	0	1	0	0
经济学	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
博弈论	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0

如果个人主页中包含该词则为1，否则为0



词-文档矩阵

	文继荣	窦志成	卢志武	徐君	魏哲巍	赵鑫	宋睿华	严睿	许洪腾	苏冰	王子贺	沈蔚然	胡迪	刘勇	毛佳昕	陈旭
信息检索	1	1	0	1	0	1	1	1	0	0	0	0	0	0	1	1
搜索	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	0
推荐	0	0	0	1	0	1	1	0	0	0	0	0	0	0	0	1
数据库	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
语言	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0
视觉	0	0	1	0	0	0	0	0	0	1	0	0	1	0	0	0
对话	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0
数据挖掘	1	1	0	1	0	1	0	0	0	0	0	0	0	1	0	0
经济学	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
博弈论	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0

哪位老师研究信息检索



词-文档矩阵

	文继荣	窦志成	卢志武	徐君	魏哲巍	赵鑫	宋睿华	严睿	许洪腾	苏冰	王子贺	沈蔚然	胡迪	刘勇	毛佳昕	陈旭
信息检索	1	1	0	1	0	1	1	1	0	0	0	0	0	0	1	1
搜索	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	0
推荐	0	0	0	1	0	1	1	0	0	0	0	0	0	0	0	1
数据库	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
语言	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0
视觉	0	0	1	0	0	0	0	0	0	1	0	0	1	0	0	0
对话	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0
数据挖掘	1	1	0	1	0	1	0	0	0	0	0	0	0	1	0	0
经济学	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
博弈论	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0

哪位老师研究计算机视觉



词-文档矩阵

	文继荣	窦志成	卢志武	徐君	魏哲巍	赵鑫	宋睿华	严睿	许洪腾	苏冰	王子贺	沈蔚然	胡迪	刘勇	毛佳昕	陈旭
信息检索	1	1	0	1	0	1	1	1	0	0	0	0	0	0	1	1
搜索	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	0
推荐	0	0	0	1	0	1	1	0	0	0	0	0	0	0	0	1
数据库	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
语言	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0
视觉	0	0	1	0	0	0	0	0	0	1	0	0	1	0	0	0
对话	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0
数据挖掘	1	1	0	1	0	1	0	0	0	0	0	0	0	1	0	0
经济学	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
博弈论	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0

哪位老师研究自然语言处理和推荐系统

词在文档中出现的向量

	文继荣	窦志成	卢志武	徐君	魏哲巍	赵鑫	宋睿华	严睿	许洪腾	苏冰	王子贺	沈蔚然	胡迪	刘勇	毛佳昕	陈旭
信息检索	1	1	0	1	0	1	1	1	0	0	0	0	0	0	1	1
搜索	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	0
推荐	0	0	0	1	0	1	1	0	0	0	0	0	0	0	0	1
数据库	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
语言	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0

哪位老师研究自然**语言**处理和**推荐**系统

- 对词，我们有一个0/1向量，表示该词是否出现在对应文档中
- 通过逐维度计算逻辑与（AND）操作，我们可以回答上述问题：
 - 00010**1**10000000001
 - 01100**1**000000000000



词-文档矩阵

	文继荣	窦志成	卢志武	徐君	魏哲巍	赵鑫	宋睿华	严睿	许洪腾	苏冰	王子贺	沈蔚然	胡迪	刘勇	毛佳昕	陈旭
信息检索	1	1	0	1	0	1	1	1	0	0	0	0	0	0	1	1
搜索	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	0
推荐	0	0	0	1	0	1	1	0	0	0	0	0	0	0	0	1
数据库	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
语言	0	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0
视觉	0	0	1	0	0	0	0	0	0	1	0	0	1	0	0	0
对话	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0
数据挖掘	1	1	0	1	0	1	0	0	0	0	0	0	0	1	0	0
经济学	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
博弈论	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0

哪位老师研究**信息检索**和**搜索**



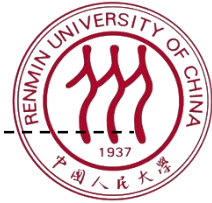
如果待检索集合变得更大

- 考虑如下情况:
 - 有100万个文档，每个文档包含大约1000词
 - 每个词大约需要两个字，8个字节来保存
 - 总共需要 $1M * 1k * 8 \text{ Byte} = 8GB$ 来保存该文档集合
- 假设其中有40万个不同的词



词-文档矩阵的大小?

- 不同的词的数量 x 文档的数量
 - $400,000 \times 1,000,000 = 4 \times 10^{11}$
 - 如果用一个字节保存0/1值, 那么共需要400GB空间
- 然而其中最多有 $1,000,000 \times 1,000 = 10^9$ 个1
 - 为什么?
- 这说明词-文档矩阵是一个非常**稀疏(sparse)**的矩阵
 - 我们可以使用一个更高效、更节约空间的方式来保存该矩阵
 - 只保存1的位置



提纲



人工智能综合设计03 信息检索和倒排索引

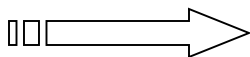
- □ 信息检索简介
- □ 词-文档矩阵
- □ 倒排索引
- □ 查询处理



倒排索引 (Inverted index)

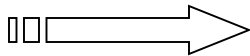
- 对于每个词t, 我们需要保存一个列表, 记录哪些文档包含该词
 - 我们用docid作为一个文档的唯一标识符
- 我们能否用一个定长的向量来保存词在哪些文档中出现呢?

信息检索



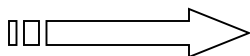
0	1	3	5	6	7	14	15
---	---	---	---	---	---	----	----

搜索



0	1	2	3	4	5	13	14
---	---	---	---	---	---	----	----

推荐

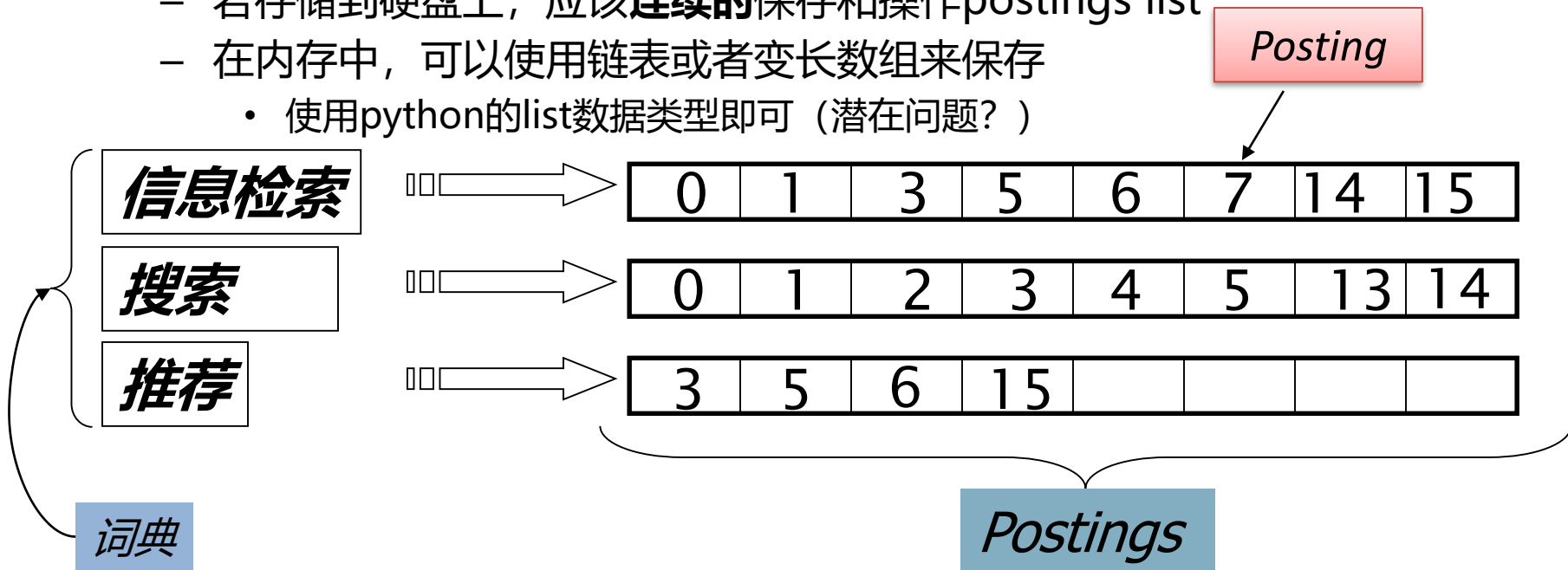


3	5	6	15				
---	---	---	----	--	--	--	--

如果要将在**搜索**这个词加入到6号文档中?

倒排索引 (Inverted index)

- 因此，我们需要使用**变长列表**来保存词出现的信息 (postings list)
 - 若存储到硬盘上，应该**连续的**保存和操作postings list
 - 在内存中，可以使用链表或者变长数组来保存
 - 使用python的list数据类型即可 (潜在问题?)



注意需要按docid顺序保存

构造倒排索引的过程

待索引文档



分词程序

分词后结果



索引程序

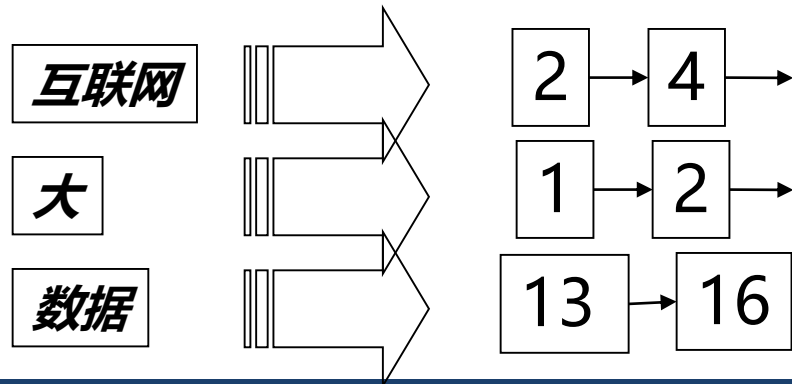
倒排索引



互联网大数据管理、信息检索（互联网搜索），

互联网 大 数据 管理

信息检索 互联网 搜索





文本处理和分词

- 分词
 - 将文本分成若干包含明确语义信息的词
- 分词结果处理
 - 去除空格
 - 去掉标点符号
 - 是否保留英文?
- 去除**停用词**(stopwords)
 - 我们**可以**不考虑频繁出现的无具体意义的词
 - 英语：冠词 (the, a)和介词(in, of, on)
 - 中文停用词表: <https://github.com/YueYongDev/stopwords>



倒排索引构建过程

- 得到 (term, docid) 对组成的序列:

Doc 0

Doc 1



互联网大数据管理、
信息检索(互联网搜索),
数据挖掘和机器学习

主要研究方向为信息检索、
网络搜索、
搜索用户行为分析
和机器学习

term	docid
互联网	0
数据管理	0
信息检索	0
互联网	0
搜索	0
数据挖掘	0
机器	0
学习	0
主要	1
研究	1
方向	1
信息检索	1
网络	1
搜索	1
搜索	1
用户	1
行为	1
分析	1
机器	1
学习	1

倒排索引构建过程

- 对其进行排序：
 - 首先按照词 (term) 进行排序
 - 然后按照docid进行排序

term	docid
互联网	0
数据管理	0
信息检索	0
互联网	0
搜索	0
数据挖掘	0
机器	0
学习	0
主要	1
研究	1
方向	1
信息检索	1
网络	1
搜索	1
搜索	1
用户	1
行为	1
分析	1
机器	1
学习	1



term	docid
主要	1
互联网	0
互联网	0
信息检索	0
信息检索	1
分析	1
学习	0
学习	1
搜索	0
搜索	1
搜索	1
数据挖掘	0
数据管理	0
方向	1
机器	0
机器	1
用户	1
研究	1
网络	1
行为	1

倒排索引构建过程

- 构建词典和postings
 - 把来自同一文档的词 (term) 合并
 - 构建词典和postings list
 - 添加词频信息(doc frequency)

term	docid
主要	1
互联网	0
互联网	0
信息检索	0
信息检索	1
分析	1
学习	0
学习	1
搜索	0
搜索	1
搜索	1
数据挖掘	0
数据管理	0
方向	1
机器	0
机器	1
用户	1
研究	1
网络	1
行为	1



term	doc frequency		postings list
主要	1	-->	1
互联网	1	-->	0
信息检索	2	-->	0->1
分析	1	-->	1
学习	2	-->	0->1
搜索	2	-->	0->1
数据挖掘	1	-->	0
数据管理	1	-->	0
方向	1	-->	1
机器	2	-->	0->1
用户	1	-->	1
研究	1	-->	1
网络	1	-->	1
行为	1	-->	1



倒排索引构建过程

词和词出现的次数

term	doc frequency		postings list
主要	1	-->	1
互联网	1	-->	0
信息检索	2	-->	0->1
分析	1	-->	1
学习	2	-->	0->1
搜索	2	-->	0->1
数据挖掘	1	-->	0
数据管理	1	-->	0
方向	1	-->	1
机器	2	-->	0->1
用户	1	-->	1
研究	1	-->	1
网络	1	-->	1
行为	1	-->	1

Docid的有序列表

指向



倒排索引构建过程

- Python实现:

```
In [187]: doc0 = "互联网大数据管理、信息检索（互联网搜索），数据挖掘和机器学习"
doc1 = "主要研究方向为信息检索、网络搜索、搜索用户行为分析和机器学习"
term_docid_pairs = [(t.strip(), 0) for t in jieba.lcut(doc0) if len(t.strip()) > 1] + \
    [(t.strip(), 1) for t in jieba.lcut(doc1) if len(t.strip()) > 1]

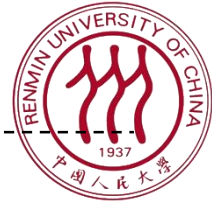
# 排序
# 默认情况下两个tuple比较大小时是先按照第一个元素比较，再按照第二个元素比较，因此不需要用key=lambda x:...的方式定义排序方式。
term_docid_pairs = sorted(term_docid_pairs)

# 构造倒排索引
# 为了编程实现方便，我们在每个postings list开头加上了一个用来标记开头的-1（正常docid从0编号）
# 注意这段代码只有在term_docid_pairs已排序的时候才是正确的
from collections import defaultdict
inverted_index = defaultdict(lambda: [-1])

for term, docid in term_docid_pairs:
    postings_list = inverted_index[term]
    if docid != postings_list[-1]:
        postings_list.append(docid)
```

```
In [186]: inverted_index
```

```
Out[186]: defaultdict(<function __main__.<lambda>()>,
    {'主要': [-1, 1],
     '互联网': [-1, 0],
     '信息检索': [-1, 0, 1],
     '分析': [-1, 1],
     '学习': [-1, 0, 1],
     '搜索': [-1, 0, 1],
     '数据挖掘': [-1, 0],
     '数据管理': [-1, 0],
     '方向': [-1, 1],
     '机器': [-1, 0, 1],
     '用户': [-1, 1],
     '研究': [-1, 1],
     '网络': [-1, 1],
     '行为': [-1, 1]})
```



提纲



人工智能综合设计03 信息检索和倒排索引

- □ 信息检索简介
- □ 词-文档矩阵
- □ 倒排索引
- □ 查询处理



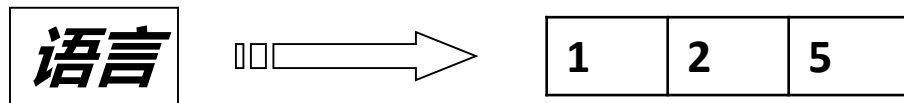
利用倒排索引处理查询

- 今天我们先关注布尔查询
 - 逻辑与AND
 - 逻辑或OR
 - 逻辑非NOT
- 考虑处理以下查询：
 - 哪位老师研究自然**语言**处理和**推荐**系统
 - 查询 (query) : **语言 AND 推荐**

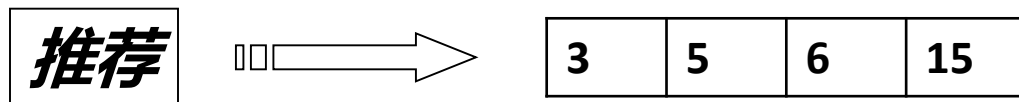
利用倒排索引处理查询

- 查询 (query) : **语言 AND 推荐**

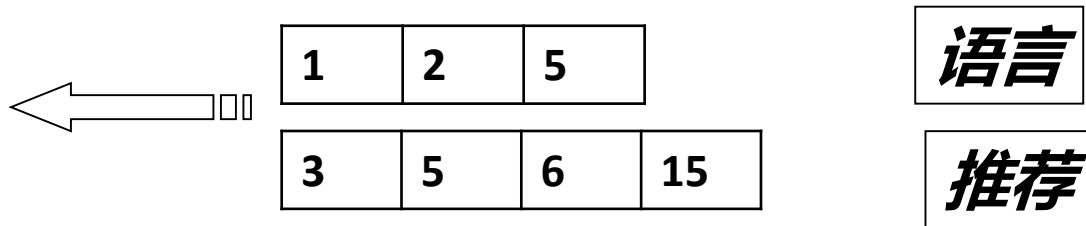
- 找到 **语言** 对应的postings list



- 找到 **推荐** 对应的postings list

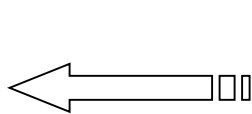


- 对两个postings list进行归并操作



利用倒排索引处理查询

- 按顺序遍历两个postings list
 - 在线性时间返回结果



1	2	5
---	---	---

3	5	6	15
---	---	---	----

语言

推荐

- 如果两个postings list的长度分别为 x 和 y ，该算法可以在 $O(x+y)$ 的时间内完成

利用倒排索引处理查询

- 求两个postings list的交集

```
INTERSECT( $p_1, p_2$ )  
1   $answer \leftarrow \langle \rangle$   
2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$   
3  do if  $docID(p_1) = docID(p_2)$   
4      then  $\text{ADD}(answer, docID(p_1))$   
5           $p_1 \leftarrow next(p_1)$   
6           $p_2 \leftarrow next(p_2)$   
7      else if  $docID(p_1) < docID(p_2)$   
8          then  $p_1 \leftarrow next(p_1)$   
9          else  $p_2 \leftarrow next(p_2)$   
10 return  $answer$ 
```




利用倒排索引处理查询

- Python实现:

```
In [184]: l1 = [1, 2, 5]
          l2 = [3, 5, 6, 15]

def intersection(l1, l2):
    answer = []
    p1, p2 = iter(l1), iter(l2)
    try:
        docID1, docID2 = next(p1), next(p2)
        while True:
            if docID1 == docID2:
                answer.append(docID1)
                docID1, docID2 = next(p1), next(p2)
            elif docID1 < docID2:
                docID1 = next(p1)
            else:
                docID2 = next(p2)
    except StopIteration:
        pass
    return answer

intersection(l1, l2)
```

```
Out[184]: [5]
```



利用倒排索引处理查询

- 思考：
 - 如何用类似的方法处理：
 - 逻辑或OR 查询
 - 逻辑非NOT 查询
 - 应该如何解析输入的查询？
 - 语言 AND 推荐
 - 语言 OR 推荐
 - 语言 AND NOT 推荐



谢谢!



- 实现倒排索引
- 基于倒排索引，支持两种布尔查询：
 - 逻辑与AND
 - 逻辑或OR
- 在昨天抓取的网页上建立倒排索引